

Model -> API -> Docker -> Kubernetes -> Google Cloud

A hands-on lab for building CV evidence across ML engineering, platform engineering, containers and cloud.

End state: A public Git repository with model training code, automated tests, a FastAPI inference service, Dockerfile, Kubernetes manifests, CI, and an optional Cloud Run endpoint that scales to zero.

Cost strategy: Run Kubernetes locally with kind. Use Google Artifact Registry only for the image and Cloud Run for the live demo. Do not create a persistent GKE cluster for this lab.

Time limit: 120 minutes hard stop. If a cloud command blocks you for more than 5 minutes, capture the error in NOTES.md and continue to the repo-polish section.

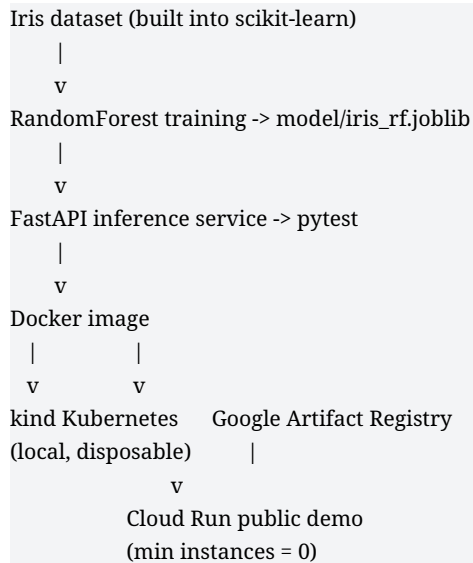
What you will be able to demonstrate

- Train and serialize a small supervised ML model with scikit-learn.
- Serve model inference through a typed REST API with health checking.
- Build and run the service as a Docker container.
- Deploy the container to a local Kubernetes cluster with probes and resource controls.
- Publish an amd64 container image to Google Artifact Registry.
- Deploy a low-idle-cost public endpoint on Cloud Run.
- Package the work as a reviewer-friendly GitHub repository with CI and measurable evidence.

120-minute run sheet

Time	Lab stage	Evidence produced	Stop condition
0-10	Scaffold + environment	Repo tree, venv	Dependencies install
10-30	Model development	Serialized model, accuracy output	Model file exists
30-45	Inference API + tests	FastAPI /health + /predict, pytest	Tests green
45-65	Docker	Runnable container image	curl succeeds in container
65-90	Kubernetes	Deployment, Service, probe, resources	Pod Ready + prediction succeeds
90-108	Google Cloud	Artifact Registry image + Cloud Run URL	Cloud endpoint responds
108-120	Portfolio polish	README, CI, screenshots/evidence, CV bullet	Repo ready to push

Architecture



Before the timer starts

Install or confirm these tools. Installation time is not counted in the 2-hour lab.

```
python3 --version
docker --version
kubectl version --client
kind version
gcloud --version
git --version
```

On macOS with Homebrew, missing CLI tools can be installed with:

```
brew install kubectl kind google-cloud-sdk
```

Apple Silicon note: Build the local kind image normally for your Mac architecture. For Cloud Run, the lab explicitly builds/pushes a linux/amd64 image.

Stage 1 - Scaffold the repository (0-10 min)

```
mkdir ml-platform-lab && cd ml-platform-lab
git init
mkdir -p src tests model k8s .github/workflows
python3 -m venv .venv
source .venv/bin/activate
```

Create requirements.txt:

```
fastapi==0.116.1
uvicorn[standard]==0.35.0
scikit-learn==1.7.1
joblib==1.5.1
pydantic==2.11.7
```

```
pytest==8.4.1
httpx==0.28.1
pip install -r requirements.txt
```

Target repository structure:

```
ml-platform-lab/
├── .github/workflows/ci.yml
├── k8s/deployment.yaml
├── k8s/service.yaml
├── model/iris_rf.joblib
├── src/api.py
├── src/train.py
├── tests/test_api.py
├── Dockerfile
├── README.md
├── requirements.txt
└── .gitignore
```

Checkpoint: You can run python, pip and git inside the new directory. Commit the empty scaffold if you want a clean project history.

Stage 2 - Train the model (10-30 min)

Use the built-in Iris dataset so the lab has no data-download dependency. Create src/train.py:

```
from pathlib import Path
import joblib
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split

MODEL_PATH = Path("model/iris_rf.joblib")
MODEL_PATH.parent.mkdir(parents=True, exist_ok=True)

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
accuracy = accuracy_score(y_test, model.predict(X_test))
joblib.dump(model, MODEL_PATH)
print(f'saved={MODEL_PATH} test_accuracy={accuracy:.3f}')

python src/train.py
```

Expected result: model/iris_rf.joblib is created and test accuracy is printed. Do not spend time tuning hyperparameters; the engineering lifecycle is the learning objective.

Professional signal: The model is deterministic because random_state is fixed, and the model artifact is produced by code rather than manually copied into the repo.

Stage 3 - Build the inference API and tests (30-45 min)

Create src/api.py:

```
from pathlib import Path
import joblib
from fastapi import FastAPI
from pydantic import BaseModel, Field

MODEL_PATH = Path("model/iris_rf.joblib")
model = joblib.load(MODEL_PATH)

app = FastAPI(title="Iris ML Platform Lab", version="1.0.0")

class IrisFeatures(BaseModel):
    sepal_length: float = Field(gt=0)
    sepal_width: float = Field(gt=0)
    petal_length: float = Field(gt=0)
    petal_width: float = Field(gt=0)

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/predict")
def predict(features: IrisFeatures):
    x = [
        features.sepal_length, features.sepal_width,
        features.petal_length, features.petal_width
    ]
    prediction = int(model.predict(x)[0])
    probabilities = model.predict_proba(x)[0].tolist()
    return {"class_id": prediction, "probabilities": probabilities}
```

Create tests/test_api.py:

```
from fastapi.testclient import TestClient
from src.api import app

client = TestClient(app)

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json() == {"status": "ok"}

def test_predict():
    response = client.post("/predict", json={
        "sepal_length": 5.1,
        "sepal_width": 3.5,
        "petal_length": 1.4,
        "petal_width": 0.2
    })
    assert response.status_code == 200
```

```
body = response.json()
assert body["class_id"] in [0, 1, 2]
assert len(body["probabilities"]) == 3

pytest -q
```

Then run the service locally:

```
uvicorn src.api:app --reload --port 8080
```

In a second terminal:

```
curl http://127.0.0.1:8080/health

curl -X POST http://127.0.0.1:8080/predict \
-H 'Content-Type: application/json' \
-d '{"sepal_length":5.1,"sepal_width":3.5,"petal_length":1.4,"petal_width":0.2}'
```

Checkpoint: pytest passes, /health returns {"status":"ok"}, and /predict returns a class_id plus three probabilities.

Stage 4 - Containerize with Docker (45-65 min)

Create Dockerfile:

```
FROM python:3.12-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY src ./src
COPY model ./model

ENV PORT=8080
EXPOSE 8080
CMD ["sh", "-c", "uvicorn src.api:app --host 0.0.0.0 --port ${PORT}"]

docker build -t ml-platform-lab:local .
docker run --rm -p 8080:8080 ml-platform-lab:local
```

From another terminal:

```
curl http://127.0.0.1:8080/health
```

- A slim base image.
- Dependency installation separated from application copy for better layer caching.
- No notebook dependency in production runtime.
- PORT environment variable compatible with Cloud Run.

Checkpoint: The API works from inside the container, not only from your Python virtual environment.

Stage 5 - Deploy to local Kubernetes with kind (65-90 min)

Create k8s/deployment.yaml:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: iris-api
spec:
  replicas: 1
  selector:
    matchLabels:
      app: iris-api
  template:
    metadata:
      labels:
        app: iris-api
    spec:
      containers:
        - name: iris-api
          image: ml-platform-lab:local
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8080
      readinessProbe:
        httpGet:
          path: /health
          port: 8080
        initialDelaySeconds: 3
        periodSeconds: 5
      resources:
        requests:
          cpu: "100m"
          memory: "128Mi"
        limits:
          cpu: "500m"
          memory: "256Mi"

```

Create k8s/service.yaml:

```

apiVersion: v1
kind: Service
metadata:
  name: iris-api
spec:
  selector:
    app: iris-api
  ports:
    - port: 80
      targetPort: 8080

kind create cluster --name ml-lab
kind load docker-image ml-platform-lab:local --name ml-lab
kubectl apply -f k8s/
kubectl get pods
kubectl rollout status deployment/iris-api
kubectl port-forward service/iris-api 8080:80

```

Test again through Kubernetes:

```

curl http://127.0.0.1:8080/health
curl -X POST http://127.0.0.1:8080/predict \

```

```
-H 'Content-Type: application/json' \  
-d '{"sepal_length":6.2,"sepal_width":2.9,"petal_length":4.3,"petal_width":1.3}'
```

Inspect the deployment like an SRE/platform engineer:

```
kubectl get deployment,service,pods  
kubectl describe deployment iris-api  
kubectl logs deployment/iris-api
```

Why this is stronger than a hello-world deployment: Your manifest includes a readiness probe plus CPU/memory requests and limits. Those are operational controls reviewers expect in production-oriented Kubernetes work.

When finished with local Kubernetes:

```
kind delete cluster --name ml-lab
```

Stage 6 - Publish and deploy on Google Cloud (90-108 min)

Set variables. Replace the project ID with your own; choose a nearby Artifact Registry/Cloud Run region.

```
export PROJECT_ID="YOUR_GCP_PROJECT_ID"  
export REGION="europe-west1"  
export REPO="ml-lab"  
export IMAGE="iris-api"  
  
gcloud auth login  
gcloud config set project "$PROJECT_ID"  
gcloud services enable artifactregistry.googleapis.com run.googleapis.com
```

Create a small Docker repository in Artifact Registry:

```
gcloud artifacts repositories create "$REPO" \  
--repository-format=docker \  
--location="$REGION" \  
--description="2-hour ML platform portfolio lab"
```

Configure Docker authentication and push a Cloud-compatible image:

```
gcloud auth configure-docker "$REGION-docker.pkg.dev"  
  
docker buildx build \  
--platform linux/amd64 \  
-t "$REGION-docker.pkg.dev/$PROJECT_ID/$REPO/$IMAGE:v1" \  
--push .
```

Deploy to Cloud Run. min=0 is the important cost guardrail:

```
gcloud run deploy iris-api \  
--image "$REGION-docker.pkg.dev/$PROJECT_ID/$REPO/$IMAGE:v1" \  
--region "$REGION" \  
--platform managed \  
--allow-unauthenticated \  
--min 0 \  
--max 1 \  
--memory 512Mi \  
--cpu 1
```

Copy the returned service URL and test /health. Then add it to the README as an optional live demo.

Cost guardrail: Cloud Run can scale to zero when idle. Artifact Registry still stores the image, so keep only one small tag or delete the repository after capturing evidence. Do not provision GKE for this 2-hour exercise.

Stage 7 - Make it recruiter-reviewable (108-120 min)

Create .gitignore:

```
.venv/  
__pycache__/  
.pytest_cache/  
.DS_Store  
*.pyc
```

Add a minimal CI workflow at .github/workflows/ci.yml:

```
name: CI  
  
on:  
  push:  
  pull_request:  
  
jobs:  
  test-and-build:  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v4  
      - uses: actions/setup-python@v5  
        with:  
          python-version: "3.12"  
      - run: pip install -r requirements.txt  
      - run: python src/train.py  
      - run: pytest -q  
      - run: docker build -t ml-platform-lab:ci .
```

Your README should answer these questions in the first screenful:

- What problem does the repo solve?
- What is the architecture?
- How do I run tests locally?
- How do I build and run the Docker image?
- How do I deploy it to Kubernetes?
- What reliability controls are present?
- Where is the optional live demo?
- How do I clean up cloud resources?

```
git add .  
git commit -m "build end-to-end ML platform lab"  
# Create an empty GitHub repo, then:  
git branch -M main  
git remote add origin <YOUR_GITHUB_REPO_URL>  
git push -u origin main
```


Definition of done

- ☐ Model training command is reproducible and generates model/iris_rf.joblib.
- ☐ pytest passes locally.
- ☐ Docker container responds to /health and /predict.
- ☐ Kubernetes pod reaches Ready state; service works through port-forward.
- ☐ Kubernetes manifest contains readiness probe and resource requests/limits.
- ☐ GitHub Actions workflow tests and builds the project.
- ☐ README contains architecture, runbook and cleanup instructions.
- ☐ Optional: Cloud Run URL responds and is linked from README.
- ☐ No credentials, gcloud config files, service-account keys or secrets are committed.

If you finish early: 10-minute SRE drill

Choose one only. Capture the command and observation in README under "Operational notes".

- Delete the pod and watch the Deployment recreate it: `kubectl delete pod <pod-name>`.
- Temporarily change /health to return an error, rebuild/redeploy and observe readiness failure.
- Set a very low memory limit and inspect the failure mode with `kubectl describe pod`.
- Scale the Deployment to two replicas and inspect endpoints: `kubectl scale deployment iris-api --replicas=2`.

How to explain this project in an interview

60-second narrative: I built a reproducible ML inference service rather than stopping at model training. I trained and serialized a scikit-learn classifier, wrapped it in a typed FastAPI service, tested it, containerized it, and deployed it to Kubernetes with readiness and resource controls. I then published the image to Artifact Registry and used Cloud Run for a low-idle-cost public demo. The repo includes CI and cleanup instructions, so the focus is the full model-to-production lifecycle.

Example CV bullet:

CV bullet: Built an end-to-end ML inference platform using Python, scikit-learn, FastAPI, Docker and Kubernetes; implemented health/readiness checks, resource controls and CI, and published a scale-to-zero Google Cloud Run demo via Artifact Registry.

Cloud cleanup

Run these when you no longer need the demo:

```
gcloud run services delete iris-api --region "$REGION"  
gcloud artifacts repositories delete "$REPO" --location "$REGION"
```

Do not delete blindly: If you reused an existing Artifact Registry repository that contains other images, delete only this image/tag instead of deleting the whole repository.

What this lab intentionally does not include

The 2-hour constraint is deliberate. A second iteration can upgrade the same repository with model versioning, structured logging, Prometheus metrics, load testing, Terraform, vulnerability scanning, GitOps, Vertex AI, or GKE Autopilot. Those additions are more valuable after this baseline works end to end.

Reference solution

A matching reference repository is supplied alongside this PDF as `ml-platform-lab-reference.zip`. Build the files yourself during the lab; use the archive only to compare your result or recover if typing/configuration consumes the time box.